SOURCE CODE WITH COMMENTS

COMMENTS:

mkdir langgraph

cd langgraph

py -3.12 -m -venv venv

.\venv\Scripts\Activate

code .

pip install langgraph langchain

SOURCE CODE (WITH ROUTER AND MEMORY CONTAINING ALL TASKS):

```
"""

ABC Technologies — AI Customer Support Automation System

Implementation Platform: LangGraph, LangChain, Ollama (Qwen2.5:3b)

"""


import os

import sqlite3

from typing import Annotated, Literal, TypedDict
```

```
from langchain_core.messages import AIMessage,
HumanMessage
from langchain_ollama import ChatOllama
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph import StateGraph, END
from langgraph.graph.message import add_messages
from rich.console import Console
from rich.panel import Panel

console = Console()

# Initialize Local Model Connection
llm = ChatOllama(model="qwen2.5:3b", temperature=0.0)
DB_PATH = "support_memory.db"

#
==========================================================
==========
# TASK 2: State Structure Design
#
==========================================================
==========
class SupportState(TypedDict):
```

```
    """Tracks state parameters across graph node traversals."""
    messages: Annotated[list, add_messages]  # Accumulates conversation logs
    current_query: str                  # Latest user string input
    category: Literal["sales", "technical", "billing", "account", "memory_recall"]
    retrieved_context: str              # Matched text from knowledge base documents
    requires_approval: bool             # Security flag for high-risk topics
    approval_status: Literal["pending", "approved", "rejected"]
    supervisor_approved: bool           # Quality check confirmation flag
    final_response: str                 # Delivered textual output string


# ============================================================
# TASK 3: Intent Classification Node
# ============================================================
def intent_classifier_node(state: SupportState) -> dict:
```

```
    """Classifies user queries into support verticals using LLM
intelligence."""
    latest_msg = state["messages"][-1].content if
state["messages"] else ""

    # Fast static routing for history-specific checks
    if any(kw in latest_msg.lower() for kw in ["previous issue",
"past request", "what did i say"]):
        return {
            "current_query": latest_msg,
            "category": "memory_recall",
            "requires_approval": False,
            "approval_status": "approved"
        }

    prompt = f"""You are a customer support query classifier
for ABC Technologies.
Classify the following query into exactly ONE category: sales,
technical, billing, or account.

Query: "{latest_msg}"

Rules:
```

- sales: Pricing, subscription tiers, product details.

- technical: Application crashes, system bugs, installation errors.

- billing: Invoices, payment errors, refund requests.

- account: Password resets, credentials, updates.

Reply with EXACTLY one word from options: [sales, technical, billing, account]. Do not write anything else.
"""

```
    response = llm.invoke([HumanMessage(content=prompt)]).content.strip().lower()


    # Extraction fallback guardrail
    category = "sales"
    for cat in ["sales", "technical", "billing", "account"]:
        if cat in response:
            category = cat
            break


    # Determine structural safety boundaries (Task 5/8 rules)
    high_risk_keywords = ["refund", "cancel", "closure", "close my account", "compensation", "escalate"]
```

```
    requires_approval = any(kw in latest_msg.lower() for kw in
high_risk_keywords)


    console.print(f"[bold cyan] 🔍 [TASK 3 - INTENT
CLASSIFICATION]:[/bold cyan] Category ->
[yellow]{category.upper()}[/yellow] | Critical Flag:
{requires_approval}")
    return {
        "current_query": latest_msg,
        "category": category,
        "requires_approval": requires_approval,
        "approval_status": "pending" if requires_approval else
"approved"
    }


#
==========================================================
==========
# TASK 4: Conditional Routing Function
#
==========================================================
==========
def route_by_intent(state: SupportState) -> str:
```

```
    """Interprets the state context to execute branch
redirection."""
    return state["category"]


#
==========================================================
==========
```

TASK 6: Knowledge Base Retrieval Automation (RAG Pipeline)

```
#
==========================================================
==========
def mock_rag_retrieval(query: str, category: str) -> str:
    """Queries specialized corporate documents based on
intent."""
    docs = {
        "sales": "[Pricing Guide]: ABC Technologies offers
standard SaaS cloud subscription structures: Starter ($29/mo)
providing fundamental access, Professional ($79/mo) with
full core capabilities, and Enterprise Custom options.",
        "account": "[FAQ / Corporate Policy]: Password reset
operations must utilize registered administrative emails.
Accounts remain locked for 30 minutes following 5 successive
structural verification failures.",
```

```
        "technical": "[Technical Manual]: File execution stack loops or upload system fault crashes are mitigated by configuring memory buffers to 512MB and forcing system cache flush cycles.",
        "billing": "[Company Policy Document]: Refund requests are processed inside a standard 14-day window following verification. All final approvals require official tier-2 human supervisor validation signs."
    }
    context = docs.get(category, "[FAQ Document]: Contact support@abctech.com for operational assistance.")
    console.print(f"  [bold magenta]📖 [TASK 6 - RAG CONTEXT RETRIEVED]:[/bold magenta] {context[:80]}...")
    return context


# ============================================================
# TASK 5: Specialized Domain Agent Nodes
# ============================================================
def sales_agent_node(state: SupportState) -> dict:
    context = mock_rag_retrieval(state["current_query"], "sales")
```

```
    prompt = f"Context: {context}\nQuery:
{state['current_query']}\nProvide a pricing summary."
    response =
llm.invoke([HumanMessage(content=prompt)]).content.strip(
)
    return {"retrieved_context": context, "final_response":
response}


def technical_agent_node(state: SupportState) -> dict:
    context = mock_rag_retrieval(state["current_query"],
"technical")
    prompt = f"Context: {context}\nQuery:
{state['current_query']}\nProvide a resolution strategy."
    response =
llm.invoke([HumanMessage(content=prompt)]).content.strip(
)
    return {"retrieved_context": context, "final_response":
response}


def billing_agent_node(state: SupportState) -> dict:
    context = mock_rag_retrieval(state["current_query"],
"billing")
    prompt = f"Context: {context}\nQuery:
{state['current_query']}\nProvide transactional details."
```

```
    response =
llm.invoke([HumanMessage(content=prompt)]).content.strip(
)
    return {"retrieved_context": context, "final_response":
response}


def account_agent_node(state: SupportState) -> dict:
    context = mock_rag_retrieval(state["current_query"],
"account")
    prompt = f"Context: {context}\nQuery:
{state['current_query']}\nProvide credential recovery setup
steps."
    response =
llm.invoke([HumanMessage(content=prompt)]).content.strip(
)
    return {"retrieved_context": context, "final_response":
response}


#
==========================================================
==========
```

TASK 7: Memory Recall Engine Node

```
#
==========================================================
==========
```

```
def memory_recall_node(state: SupportState) -> dict:
    """Uses conversational historical context stored via SQLite checkpoints."""
    history_events = []
    for msg in state["messages"][:-1]:  # Read historical tracking messages
        role = "User" if isinstance(msg, HumanMessage) else "Support Agent"
        history_events.append(f"{role}: {msg.content}")

    summary = "\n".join(history_events) if history_events else "No previous support records tracking found."
    prompt = f"History Logs:\n{summary}\nCurrent Question: {state['current_query']}\nAnswer based on logs."
    response = llm.invoke([HumanMessage(content=prompt)]).content.strip()

    console.print("  [bold blue] ⏱️  [TASK 7 - SQLITE MEMORY RECALL EXECUTED][/bold blue]")
    return {"final_response": response}
```

```
#
==========
```

TASK 8: Human-In-The-Loop Approval Node & Router

```
#
==========
def human_approval_router(state: SupportState) -> str:
    """Routes high-risk financial/data operations to supervisor
desk."""
    if state["requires_approval"] and state["approval_status"]
== "pending":
        return "human_checkpoint"
    return "supervisor_agent"


def human_checkpoint_node(state: SupportState) -> dict:
    """Creates a manual evaluation checkpoint prompt
block."""
    console.print("\n[bold orange3] ⚠️  [TASK 8 - HUMAN
INTERVENTION COMPLIANCE ACTION REQUIRED] ⚠️ [/bold
orange3]")
    console.print(f" Escalation Target:
[italic]{state['current_query']}[/italic]")
```

```
    choice = input("  Authorize processing approval? (yes/no): ").strip().lower()

    if choice in ["yes", "y"]:
        return {"approval_status": "approved"}
    return {
        "approval_status": "rejected",
        "final_response": "Transaction blocked: Request declined by corporate supervisor compliance."
    }

# ================================================================
# TASK 9: Supervisor Quality Agent Node
# ================================================================
def supervisor_agent_node(state: SupportState) -> dict:
    """Polishes agent text to guarantee professional style uniformity."""
    if state["approval_status"] == "rejected":
        return {"supervisor_approved": True}
```

```
    raw_text = state["final_response"]
    prompt = f"Perfect this corporate message for style compliance, removing raw fragments:\n\n{raw_text}"
    refined = llm.invoke([HumanMessage(content=prompt)]).content.strip()

    console.print("  [bold green] 🛡️  [TASK 9 - SUPERVISOR VALIDATION SYSTEM APPROVED][/bold green]")
    return {"final_response": refined, "supervisor_approved": True}

# ==========================================================
# TASK 1 & 4: LangGraph Workflow Architecture Definition
# ==========================================================
def build_workflow_graph(db_path: str):
    builder = StateGraph(SupportState)

    # Core Processing Registrations
    builder.add_node("classifier", intent_classifier_node)
```

```
    builder.add_node("sales_agent", sales_agent_node)
    builder.add_node("technical_agent",
technical_agent_node)
    builder.add_node("billing_agent", billing_agent_node)
    builder.add_node("account_agent", account_agent_node)
    builder.add_node("memory_agent", memory_recall_node)
    builder.add_node("human_checkpoint",
human_checkpoint_node)
    builder.add_node("supervisor_agent",
supervisor_agent_node)

    # Connect Sequential Nodes
    builder.set_entry_point("classifier")

    # Task 4 Conditional Framework Integration
    builder.add_conditional_edges(
        "classifier",
        route_by_intent,
        {
            "sales": "sales_agent",
            "technical": "technical_agent",
            "billing": "billing_agent",
            "account": "account_agent",
```

```
        "memory_recall": "memory_agent"
    }
)

for standard_agent in ["sales_agent", "technical_agent", "account_agent"]:
    builder.add_edge(standard_agent, "supervisor_agent")

# Task 8 Human Checkpoint Mapping
builder.add_conditional_edges(
    "billing_agent",
    human_approval_router,
    {"human_checkpoint": "human_checkpoint", "supervisor_agent": "supervisor_agent"}
)

builder.add_edge("human_checkpoint", "supervisor_agent")
builder.add_edge("memory_agent", "supervisor_agent")
builder.add_edge("supervisor_agent", END)

# Task 7 SQLite Persistence Connection
```

```
    conn = sqlite3.connect(db_path,
check_same_thread=False)
    return builder.compile(checkpointer=SqliteSaver(conn))


#
==========================================================
==========
```

 # TASK 10: System Demonstration Simulation Loop

```
#
==========================================================
==========
if __name__ == "__main__":
    if os.path.exists(DB_PATH):
        os.remove(DB_PATH)  # Refresh memory trace files for
clean execution proof


    system = build_workflow_graph(DB_PATH)
    config = {"configurable": {"thread_id":
"session_david_001"}}


    sample_queries = [
        "What are the pricing plans available for your software?",
        "I forgot my account password.",
        "My application crashes whenever I upload a file.",
```

```
        "I need a refund for my annual subscription.",
        "What was my previous support issue?"
    ]

    # FIXED: Style markup strings broken into explicit single-line pairs to avoid rich closing errors.
    console.print("\n[bold magenta]==========[/bold magenta]")
    console.print("[bold magenta] 🚀 ABC TECHNOLOGIES CUSTOMER AUTOMATION ENTERPRISE RUN[/bold magenta]")
    console.print("[bold magenta]==========[/bold magenta]")

    for query in sample_queries:
        console.print(f"\n[bold white] 📩 Customer Input:[/bold white] {query}")

        # Execute workflow transition loops
        output_state = system.invoke({"messages": [HumanMessage(content=query)]}, config=config)
```

```
    response_text = output_state.get("final_response",
"Processing Error.")


    # Save output state back to thread historical ledger
checkpointer
    system.update_state(config, {"messages":
[AIMessage(content=response_text)]})


    console.print(Panel(
        response_text,
        title=f"[bold green]TASK 10 OUTPUT - FINAL RESPONSE
({output_state['category'].upper()})[/bold green]",
        border_style="green"
    ))
```

To run:

py solution.py